

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 3**

**COURSE CODE:** CSA51 **COURSE NAME:** Cryptography and Network Security

---

### **COURSE OUTCOME COVERED**

**CO4:** *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

*Assessment Tool Weightage: 10%*

---

**QUESTIONS: 5**

**TOTAL MARKS: 25**

### **Annexure A**

#### ***Comparative Analysis***

---

#### ***Code of Conduct:***

*I SOFIA RV - 192511055 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***  
SOFIA RV - 192511055

## *Annexure A*

### *Comparative Analysis*

1. Compare Authentication Header (AH) and Encapsulating Security Payload (ESP) in IPSec with respect to security services, performance overhead, and suitability for different network scenarios. Analyze which is more appropriate for a VPN deployment.

#### **Comparison of AH and ESP in IPSec**

##### **Security Services**

The Authentication Header (AH) and Encapsulating Security Payload (ESP) are the two core protocols of the IPSec suite, and they differ fundamentally in the security services they offer. AH provides connectionless integrity, data origin authentication, and optional anti-replay protection, but it does not provide confidentiality at all — it never encrypts the payload. What makes AH distinctive is the scope of what it authenticates: it computes its integrity check value over the entire IP packet, including most fields of the outer IP header (excluding only mutable fields such as TTL). This means AH can detect if the source or destination address of a packet has been tampered with in transit.

ESP, by contrast, provides confidentiality through encryption of the payload, along with optional integrity, data origin authentication, and anti-replay protection when configured with an authentication mechanism. However, ESP's authentication coverage is narrower than AH's — it authenticates only the ESP header, payload, and trailer, not the outer IP header. This narrower scope is actually a deliberate design choice, because it allows ESP to function correctly even when the outer IP header is altered by intermediate devices such as NAT gateways, whereas AH's inclusion of the outer header in its integrity calculation makes it break under NAT translation.

##### **Performance Overhead**

From a performance standpoint, AH is generally lighter-weight because it only requires computing a cryptographic hash (such as HMAC-SHA) over the packet, without any encryption or decryption workload. ESP, when providing confidentiality, must additionally perform symmetric encryption and decryption of the payload, which historically imposed a heavier computational burden. However, this gap has narrowed considerably in modern deployments due to hardware-accelerated cryptography (such as AES-NI instructions in modern CPUs), meaning the throughput difference between AH and ESP is often negligible in practice. In terms of packet overhead, ESP typically adds a somewhat larger footprint than AH because it must accommodate an ESP header, an initialization vector, padding, and a trailer with authentication data, whereas AH adds a comparatively compact fixed-size header.

##### **Suitability for Different Network Scenarios**

The choice between AH and ESP depends heavily on the trust characteristics of the network path and the presence of intermediate devices. AH is best suited to scenarios where data confidentiality is not a concern but strong, whole-packet origin authentication is required — for example, authenticating routing

protocol updates within a trusted internal network where payload data is not sensitive and no NAT devices exist along the path. ESP is better suited to scenarios that traverse untrusted or public networks, where payload confidentiality is essential and NAT traversal is likely, such as communication between geographically distributed offices over the public internet.

#### Analysis: Appropriateness for VPN Deployment

When evaluating which protocol is more appropriate for a VPN deployment, ESP emerges as clearly the more suitable choice, and this can be reasoned through three lines of argument. First, a VPN's core purpose is to protect data as it crosses an inherently untrusted medium — the public internet — which makes confidentiality a fundamental requirement, and only ESP is capable of providing this; AH's inability to encrypt payload data disqualifies it as a sole protocol for any VPN carrying sensitive information. Second, real-world VPN endpoints are frequently situated behind NAT devices, whether at a branch office's edge router or a remote worker's home router, and AH's dependence on validating the unmodified outer IP header makes it fundamentally incompatible with NAT unless elaborate workarounds are used. ESP, especially when combined with NAT-Traversal (encapsulating ESP within UDP), was specifically designed to function correctly in these conditions. Third, when ESP is configured with an integrated authentication mechanism (such as HMAC-based authentication, or an authenticated-encryption cipher like AES-GCM), it already delivers integrity, data origin authentication, and anti-replay protection alongside confidentiality — meaning it can independently satisfy nearly all the security objectives a VPN requires without needing to be paired with AH.

For these reasons, ESP in tunnel mode, ideally using an authenticated-encryption algorithm, represents the theoretically and practically appropriate protocol choice for VPN deployment, while AH's role is more appropriately confined to specialized scenarios demanding whole-header authentication without confidentiality, in trusted, non-NATed network segments.

2. Compare transport mode and tunnel mode in IPSec architecture. Evaluate their effectiveness in securing host-to-host versus network-to-network communication.

### Comparison of Transport Mode and Tunnel Mode in IPSec Architecture

#### Fundamental Structural Difference

IPSec can operate in two distinct modes — transport mode and tunnel mode — and the essential difference between them lies in what portion of the original IP packet is protected and whether a new IP header is introduced. In transport mode, only the payload of the original IP packet (the transport-layer segment, such as TCP or UDP data) is protected — either encrypted, authenticated, or both — while the original IP header remains largely intact and unencrypted, simply having the AH or ESP header inserted between the IP header and the transport-layer payload. Because the original header is preserved, the source and destination addresses visible on the wire are the actual communicating endpoints themselves.

In tunnel mode, by contrast, the entire original IP packet — header and payload together — is encapsulated and protected as a single unit, and a completely new outer IP header is added on top. This outer header typically reflects the addresses of the security gateways (such as VPN routers or firewalls) performing the encapsulation, rather than the original communicating hosts. The original inner header, being fully encapsulated, is hidden from anyone observing traffic on the network between the gateways.

### **Security Implications of This Difference**

This structural distinction carries direct security consequences. Transport mode protects the data being carried but does not conceal who is communicating with whom, since the real source and destination IP addresses remain visible in the outer header. Tunnel mode, because it wraps the entire original packet inside a new one, provides an additional layer of protection by concealing the internal network topology and the true endpoints of communication from any observer along the path between the gateways — an attacker monitoring the link only sees gateway-to-gateway traffic, not the underlying host addresses.

### **Overhead Considerations**

Tunnel mode inherently introduces more overhead than transport mode because it must carry an entirely additional IP header alongside the original one, whereas transport mode only inserts the AH or ESP header without duplicating the IP header. This makes transport mode marginally more efficient in terms of bandwidth consumption, though the difference is generally small relative to overall packet size, particularly for larger payloads.

### **Effectiveness for Host-to-Host Communication**

For host-to-host communication — where two specific endpoints, such as two servers or a client and a server, wish to secure a direct connection between themselves — transport mode is generally the more natural and effective fit. Since both communicating parties are IPSec-aware endpoints and the goal is simply to protect the payload exchanged directly between them, there is little practical benefit in adding an entirely new outer header purely to hide addresses that both endpoints already know and that may not require concealment, such as within a single trusted internal network segment. Transport mode achieves the necessary confidentiality and integrity for this direct relationship with lower overhead, and is well suited to protecting sensitive application data exchanged between two specific, mutually aware hosts.

### **Effectiveness for Network-to-Network Communication**

For network-to-network communication — where the objective is to secure traffic flowing between two entire networks, such as two branch offices connected via their respective security gateways — tunnel

mode is decisively more effective and is, in practice, the standard choice. In this scenario, the actual communicating hosts (individual workstations or servers within each branch) are typically unaware that IPSec is even being applied; the security gateways at the network edges perform encryption and decapsulation transparently on their behalf. This requires that the entire original packet, including its internal, potentially private (non-routable) addressing, be hidden and carried securely across the public segment between gateways — a requirement only tunnel mode fulfills, since it fully encapsulates the original packet inside a new one addressed between the gateways rather than the internal hosts.

## **Evaluative Summary**

In evaluating effectiveness across these two use cases, the underlying principle is that the appropriate mode should match the trust boundary being protected. Transport mode is architecturally suited to protecting a specific end-to-end relationship between two IPSec-capable hosts that already trust each other's identities and do not require topology concealment, making it efficient and adequate for host-to-host scenarios, particularly within a single administrative or trusted domain. Tunnel mode is architecturally suited to protecting traffic that must cross an untrusted intermediate network on behalf of many underlying hosts that are not themselves IPSec-aware, making it the necessary and effective choice for network-to-network (site-to-site) VPN scenarios, since it both encrypts the full original packet and conceals the internal addressing structure of each network from the public path between them. In large enterprise deployments, both modes may even be combined — tunnel mode between gateways for network-to-network protection, nested with transport mode between specific sensitive hosts for defense-in-depth — reflecting that the two modes address complementary rather than competing security needs.

3. Perform a comparative analysis of Pretty Good Privacy (PGP) and S/MIME in terms of key management, trust models, scalability, and enterprise usability. Recommend the most suitable approach for a corporate email system.

## **Comparative Analysis of PGP and S/MIME**

### **Overview**

Pretty Good Privacy (PGP) and Secure/Multipurpose Internet Mail Extensions (S/MIME) are the two dominant standards for securing email through encryption and digital signatures, and while both aim to achieve confidentiality, integrity, authentication, and non-repudiation for email communication, they differ substantially in their underlying trust architecture, key management approach, and suitability for large organizations.

## **Key Management**

PGP's key management is inherently decentralized and user-driven. Each individual generates their own public-private key pair, and there is no mandatory central authority responsible for issuing, distributing, or validating keys. Users are typically responsible for publishing their public keys — whether via public keyservers, personal websites, or direct exchange — and for managing their own key lifecycle, including revocation, which requires the user to generate and distribute a revocation certificate themselves. This places significant administrative burden on individual users rather than a central body.

S/MIME's key management, by contrast, is built around a formal Public Key Infrastructure (PKI). Certificates are issued by a trusted Certificate Authority (CA), which validates the identity of the requester before binding their public key to their identity in an X.509 certificate. Key issuance, renewal, and revocation are handled through standardized, centrally governed processes (such as Certificate Revocation Lists or the Online Certificate Status Protocol), removing much of the key-handling burden from individual end users and placing it under organizational control.

## **Trust Models**

The two systems differ most fundamentally in their trust models. PGP employs what is known as the "Web of Trust," a decentralized peer-to-peer model in which trust in a given public key is established through endorsements — users cryptographically sign one another's keys to vouch for their authenticity, and trust propagates informally through chains of these signatures. This model is flexible and does not depend on any single authority, but it also lacks a formal, auditable mechanism for establishing trust at scale, and its reliability depends on the diligence of individual users in verifying identities before signing keys.

S/MIME employs a hierarchical trust model rooted in a chain of certificate authorities, from a trusted root CA down through intermediate CAs to individual end-entity certificates. Trust in any given certificate can be validated by systematically verifying the entire chain up to a root that is already trusted by the relying party's system. This hierarchical, centrally anchored structure provides a more formal and auditable basis for establishing trust, which aligns well with organizational accountability and compliance requirements.

## **Scalability**

In terms of scalability, S/MIME is considerably better suited to large organizational deployments. Because certificate issuance and validation are centralized through a CA (or a chain of CAs), an organization can systematically provision, renew, and revoke certificates for thousands of employees

through automated enrollment processes, without depending on informal peer endorsement. PGP's Web of Trust, while workable for small groups or communities of technically sophisticated users, becomes increasingly unwieldy as the number of participants grows, since establishing sufficient trust paths between large numbers of previously unconnected users requires considerable manual effort and does not scale cleanly to an enterprise with thousands of employees and external correspondents.

### **Enterprise Usability**

From a usability standpoint, S/MIME holds a substantial practical advantage in the enterprise context because support for it is natively built into virtually all major commercial email clients and platforms, including Microsoft Outlook, Apple Mail, and enterprise messaging systems such as Microsoft Exchange, allowing certificates to be deployed and used without requiring end users to install additional software. PGP, on the other hand, typically requires third-party plugins or standalone applications to integrate with mail clients, and demands a greater degree of technical understanding from end users — such as key generation, key exchange, and web-of-trust management — which increases training overhead and the likelihood of user error in a non-technical corporate workforce.

### **Recommendation for a Corporate Email System**

Considering these four dimensions together, S/MIME is the more suitable approach for a corporate email system. Its centralized, CA-based key management integrates naturally with an organization's existing PKI and identity infrastructure; its hierarchical trust model provides the auditable, accountable trust chain that regulatory and compliance frameworks typically require; its centralized architecture scales efficiently to large numbers of employees and external business correspondents; and its native support in mainstream enterprise mail clients minimizes user training and support overhead. PGP remains a valuable option in contexts such as open-source communities, independent security research, or communication with technically sophisticated external parties who specifically require it, but for the standardized, scalable, and auditable needs of a corporate email environment, S/MIME represents the more appropriate and practically deployable solution.

4. Compare rule-based and machine learning-based email spam detection techniques. Analyze their effectiveness in handling evolving spam patterns and minimizing false positives.

## **Comparison of Rule-Based and Machine Learning-Based Spam Detection Techniques**

### **Overview**

Email spam detection systems generally fall into two broad categories: rule-based techniques, which rely on predefined, human-authored criteria to classify messages, and machine learning-based techniques, which rely on statistical models trained on data to infer patterns distinguishing spam from legitimate email. Both approaches share the same underlying objective — accurately separating unwanted or malicious messages from genuine correspondence — but they differ significantly in how that classification is achieved and in how well each adapts over time.

### **Rule-Based Techniques**

Rule-based spam detection operates through a set of explicitly defined conditions crafted by administrators or security vendors, such as keyword blacklists, sender IP blocklists, regular-expression pattern matching against known spam phrases, header-based checks (for example, verifying SPF, DKIM, or DMARC alignment), and simple heuristics like flagging excessive capitalization or suspicious attachment types. These rules are typically maintained in a rule set that is manually or semi-automatically updated as new spam campaigns are identified. The strength of rule-based systems lies in their transparency and predictability: administrators can precisely understand and audit why a given message was flagged, and rules can be deployed with immediate effect once written, without requiring a training phase.

However, rule-based systems suffer from an inherent rigidity. Because their logic is static and explicitly authored, they can only detect patterns that have already been anticipated by whoever wrote the rules. Spammers routinely evade such systems through simple obfuscation techniques — deliberate misspellings, character substitution, embedding text within images rather than as plain text, or using previously unseen sending domains — all of which can circumvent rules that were written around known indicators. As a result, rule-based systems tend to lag behind rapidly evolving spam campaigns, since each new evasion technique requires a manual update to the rule set before it can be detected.

### **Machine Learning-Based Techniques**

Machine learning-based spam detection instead relies on statistical models — such as Naïve Bayes classifiers, support vector machines, gradient-boosted decision trees, or more recently, deep learning and transformer-based models — that are trained on large corpora of labeled spam and legitimate email. Rather than relying on explicitly authored rules, these models learn to recognize complex, often subtle statistical patterns across many features simultaneously, including word frequency distributions,



structural characteristics of the message, sender behavior over time, and contextual language patterns. Because the model generalizes from training data rather than matching fixed rules, it is often able to detect novel spam variants that share underlying statistical characteristics with previously seen spam, even if the exact wording or format has changed.

The principal advantage of machine learning approaches is their adaptability: as new spam data becomes available, models can be retrained or incrementally updated to reflect evolving patterns, allowing detection capability to keep pace with attacker innovation more effectively than manually maintained rule sets. However, this adaptability comes with trade-offs. Machine learning models require substantial volumes of accurately labeled training data to perform well, can be computationally more expensive to train and deploy than simple rule evaluation, and function to some extent as "black boxes" — making it harder for administrators to explain precisely why a given message was classified as spam, which can complicate auditing and troubleshooting. Additionally, machine learning models are themselves susceptible to adversarial manipulation, where attackers deliberately craft messages designed to evade the statistical patterns a model has learned to recognize.

### **Effectiveness in Handling Evolving Spam Patterns**

When evaluated specifically against the challenge of evolving spam patterns, machine learning-based techniques demonstrate a clear theoretical advantage over purely rule-based systems. Because spam campaigns continuously change their wording, structure, and delivery techniques to evade detection, a system that can generalize from underlying statistical patterns — rather than depending on exact matches to known indicators — is inherently better positioned to catch previously unseen variants of spam that share deeper similarities with known examples. Rule-based systems, by contrast, are fundamentally reactive; they can only be updated after a new spam pattern has been identified and manually codified into a rule, creating an inevitable detection lag during which novel campaigns can succeed.

### **Effectiveness in Minimizing False Positives**

The comparison regarding false positives is more nuanced. Rule-based systems, when carefully and conservatively authored, can achieve very low false-positive rates for the specific patterns they are designed to catch, since a rule either matches or it does not — there is no ambiguity in the classification for known cases. However, because rules are often written broadly to catch as much spam as possible, poorly tuned rule sets can also generate significant false positives when legitimate email happens to share superficial characteristics with spam indicators, such as legitimate marketing emails containing common promotional keywords.

Machine learning models, when well-trained on sufficiently representative and balanced data, are generally capable of achieving a more favorable balance between detection rate and false-positive rate, because they weigh many features probabilistically rather than triggering on any single matching condition. A message need not perfectly avoid every suspicious characteristic to be classified as legitimate, since the model considers the overall pattern rather than binary rule matches. That said, machine learning models are not inherently immune to false positives — if the training data is unrepresentative, imbalanced, or does not reflect the organization's actual legitimate mail patterns, models can develop systematic biases that misclassify certain types of legitimate correspondence, and such errors can be harder to diagnose and correct than adjusting a single explicit rule.

## **Overall Assessment**

In summary, rule-based techniques offer transparency, predictability, and immediate deployability, but are structurally limited in their ability to keep pace with rapidly evolving spam tactics and can produce inconsistent false-positive behavior depending on how conservatively they are authored. Machine learning-based techniques offer superior adaptability to novel and evolving spam patterns and generally achieve a more favorable overall balance of detection accuracy and false-positive minimization, at the cost of reduced interpretability, greater computational and data requirements, and exposure to adversarial evasion. In practice, most modern, effective spam-filtering systems adopt a hybrid approach — using rule-based checks (particularly for authentication-based indicators such as SPF/DKIM/DMARC) as a fast, transparent first layer, combined with a machine learning model as a secondary, adaptive layer for nuanced content and behavioral analysis — thereby drawing on the complementary strengths of both approaches rather than relying exclusively on either one.

5. Compare manual key management and automated key management (e.g., using IKE) in IPSec. Evaluate their impact on security, scalability, and operational complexity in large organizations.

## **Comparison of Manual Key Management and Automated Key Management (IKE) in IPSec Overview**

Establishing IPSec Security Associations (SAs) requires that both communicating parties possess matching cryptographic keys and agreed-upon security parameters. This can be achieved either through manual key management, where an administrator directly configures the keys and parameters on each device, or through automated key management, most commonly implemented via the Internet Key Exchange (IKE) protocol, which dynamically negotiates keys and parameters between peers without

requiring an administrator to specify the keys themselves. These two approaches differ substantially in their security properties, scalability, and the operational burden they place on an organization.

### **Manual Key Management**

In manual key management, an administrator directly configures a shared secret key (or, in some cases, static key material for AH/ESP) on both IPSec endpoints, and this same key remains in place until it is manually changed. The primary appeal of this approach is its simplicity — it requires no supporting infrastructure such as a certificate authority or a running key-exchange protocol, and for a very small number of point-to-point connections, it can be configured quickly.

However, manual key management carries substantial security weaknesses. Because the key is set once and does not change unless an administrator intervenes, the same key material may remain in use for extended periods, increasing the window of exposure if it is ever compromised and providing no forward secrecy — meaning that if the static key is ever obtained by an attacker, all past and future traffic encrypted with it can be decrypted. Manual key management also depends entirely on the key being transmitted to both endpoints through some secure out-of-band channel, which itself introduces risk if that channel is not adequately protected, and any error in transcription or synchronization between endpoints — a common practical source of the mismatched-key problems that motivated the earlier key management discussion — will cause the SA to fail outright.

### **Automated Key Management (IKE)**

Automated key management, implemented through IKE (now standardized as IKEv2), instead allows the two IPSec peers to dynamically negotiate session keys through a cryptographic key-exchange protocol, typically based on Diffie–Hellman, without either party needing to transmit the actual key material to the other. IKE also handles authentication of the peers (via pre-shared keys, digital certificates, or other credentials), negotiation of the cryptographic algorithms and parameters to be used, and automatic renegotiation of fresh keys before existing Security Associations expire.

This automated approach provides several inherent security advantages. Because IKE performs a fresh Diffie–Hellman exchange for each new session (and can be configured to use Perfect Forward Secrecy), the compromise of one session's key material does not expose the keys used in prior or subsequent sessions. Automated rekeying before SA expiry also ensures that key material does not remain static for extended periods, substantially reducing the exposure window compared to manual key management. Furthermore, because IKE handles peer authentication and negotiation programmatically, it removes the risk of human transcription error inherent in manually distributing shared keys.

### **Impact on Security**

Evaluating the security impact of these two approaches, automated key management via IKE is decisively more secure than manual key management. Manual key management's static, long-lived keys and lack of forward secrecy represent significant vulnerabilities in any environment facing a non-trivial threat model, whereas IKE's dynamic negotiation, periodic rekeying, and support for forward secrecy

directly address these weaknesses. Manual key management may only be considered acceptable in very limited, low-risk scenarios, such as temporary testing configurations, precisely because it lacks the cryptographic hygiene that automated approaches provide by design.

### **Impact on Scalability**

The scalability implications are equally significant. Manual key management requires an administrator to individually configure and track a key for every single IPSec connection, and this burden grows at least linearly — and in fully-meshed topologies, quadratically — with the number of endpoints involved. In a large organization with dozens or hundreds of branch offices, remote users, or partner connections, manually managing keys across all these relationships becomes administratively unmanageable and highly error-prone. Automated key management scales far more gracefully, since IKE allows new peers to establish SAs dynamically the moment they are authenticated (particularly when combined with a certificate-based PKI), without requiring an administrator to individually provision and distribute a unique key for every possible pairing of endpoints. This makes automated key management a practical necessity, rather than merely a convenience, once an organization's IPSec deployment grows beyond a handful of connections.

### **Impact on Operational Complexity**

From an operational standpoint, manual key management is conceptually simple to set up initially but becomes operationally burdensome over time, because every key rotation, every new connection, and every revocation must be handled manually, and any mismatch between endpoints causes connectivity failures that must be diagnosed and corrected by hand — a recurring source of downtime in real deployments. Automated key management via IKE requires greater upfront complexity to establish — configuring IKE policies, potentially deploying a supporting PKI for certificate-based authentication, and ensuring consistent policy configuration across all gateways — but once established, it substantially reduces ongoing administrative overhead, since rekeying, negotiation, and (with appropriate PKI integration) certificate renewal and revocation can all be automated, freeing administrators from continual manual intervention.

### **Overall Evaluation**

Taken together, while manual key management may appear simpler for a very small number of static, long-term connections, it is fundamentally unsuited to the security expectations and operational realities of large organizations, given its weak forward secrecy, static key exposure, and unscalable administrative burden. Automated key management via IKE, despite requiring greater initial setup complexity and supporting infrastructure, provides substantially stronger security guarantees, scales effectively to large and dynamic network topologies, and ultimately reduces long-term operational complexity by automating the ongoing tasks of key negotiation, rotation, and renewal. For any large organization deploying IPSec across multiple sites, remote users, or partner connections, automated key management is therefore the appropriate and, in practice, the necessary approach.

### RUBRICS FOR CALCULATION:

| Criteria                | Weightage (%) | Level 4 (Exemplary)                                                           | Level 3 (Proficient)                            | Level 2 (Developing)                       | Level 1 (Beginning)                      |
|-------------------------|---------------|-------------------------------------------------------------------------------|-------------------------------------------------|--------------------------------------------|------------------------------------------|
| Scope of Items          | 20%           | Selects highly relevant items with clear scope and justification              | Selects appropriate items with minor gaps       | Selection is limited or partially relevant | Items are unclear or not relevant        |
| Comparison Depth        | 25%           | Provides detailed and comprehensive comparison across multiple aspects        | Provides adequate comparison with some depth    | Limited comparison with few aspects        | Very minimal or no comparison            |
| Use of Evidence         | 20%           | Supports comparison with accurate data, examples, or references               | Uses some supporting evidence with minor gaps   | Limited or weak supporting evidence        | No supporting evidence provided          |
| Critical Thinking       | 20%           | Demonstrates strong critical thinking with clear interpretation and reasoning | Shows reasonable interpretation with minor gaps | Basic interpretation; lacks depth          | No meaningful interpretation             |
| Clarity of Presentation | 15%           | Well-organized, clear, and logically structured presentation                  | Generally clear with minor issues               | Somewhat unclear or poorly structured      | Disorganized and difficult to understand |